

Web Dev Checklist

1.1. Develop a React component that represents the login screen

- The login screen (react component) consumes the login API that is going to be built in the back-end part of the application.
- The component displays messages it receives from the API (*i.e. Password is incorrect*).
- When the admin user is logged in, the component should redirect to the dashboard page.
- When an unauthorized user tries to reach the administrator dashboard page, the application should redirect to the login screen.

1.2. Develop an administrative dashboard from where the admin can do the following:

- View a table with an overview of all (calendar) events that are in the system.
- Admin can use a form to submit a new (calendar) event to a POST endpoint, where the endpoint will be later implemented on the back-end.
- Admin can edit events via an UPDATE endpoint, where the endpoint will be later implemented on the back-end.
- Admin can delete events (a pop-up confirms the deletion before actually deleting the event from the back-end).
- Admin should be able to view a list of attendees that are signed up for an event.

1.3. Homepage of the application should show a list of available events

- The list should only show the events that are in the future.
- When you click an event, application takes you to a page containing more details about that specific event.
- Homepage is only visible for logged in users, if the user is not logged in, the application should redirect to the login screen.

1.4. Attend a specific event via a form

- Form matches the structure of the POST endpoint (that is going to be implemented in the back-end phase) to track attendance for the event.
- Users can delete events from their calendar.

1.5. Logged in users can book a room for a meeting (Younes)

- The user can decide to book an available room if available for a particular day/time range.
- Overview of room availabilities will change accordingly as they are booked.

1.6. Multi-Page Navigation with React Router

- Navigation between pages updates the URL to reflect the current view.
- When visiting a specific URL (copy-pasting to URL bar of the browser), the correct component/page loads.
- URL parameters (e.g., filters, IDs) are supported and passed correctly through React Router to the API endpoints.

1.7. User can specify its attendance in the office (Gavin & Jay)

- Design and implement a user interface that allows users to specify their attendance.
- The UI design is up to you – choose a solution that you wish.
- Attendance data should be stored and retrievable (data persistence part is going to be implemented in the back-end side).

2.1. The student needs to implement a login system for an admin user. This will lay the basic fundament for authorization to our application.

- Login system will consist of a POST call that receives a username and password and checks if the password is correct with what is in the database. This endpoint will also register a session on the server.
- POST endpoint returns a success message if the password is correct, else it returns a reasonable error message of what went wrong.
- Create a GET endpoint that returns a Boolean value based on if the session is registered or not. Additionally, return the name of the admin user that is logged in.
- The login logic and endpoints need to be separated into a Service and a Controller.

2.2. The student needs to create a CRUD (Create, Read, Update, Delete) API controller for the Event entity. This is so that the admin can manage events.

- The Read operation needs to be a GET endpoint and must be public. This endpoint will also be used by the front-end to display an overview of events.
- Include the reviews and attendees to the event entity in the response of the GET operation.
- The other endpoints (Create, Update and Delete) needs to be protected and requires an admin to be logged in to continue with the execution of the method
- The following methods are used to implement the CRUD operations inside the Controller:
Create => POST, Read => GET, Update => PUT, Delete => DELETE.
- The JSON body of the Create endpoint needs to contain the following attributes: Title, description, date, start time, end time and location.
- Make it possible to attach a review to a specific event.

2.3. Extend the login service in such a way that it can handle multiple roles.

- A normal user/employee must be able to login to the application.
- A user must register in order to use the application, registration can be handled by the login controller.
- There must be a clear distinction between admin and user roles. The user can only view events and eventually attend them. Only an admin can create, update and delete events.

2.4. Make a POST endpoint inside a new controller that allows a logged in user to attend a new event.

- The JSON body to submit an attendance should contain a user id and an event id.
- Endpoint returns the event that the user has attended.
- Endpoint is responsible to check the availability of the event based on the date and start time.
 - o If the above condition fails, the endpoint needs to provide an error message of what went wrong.

Check event for availability

- Inside the same controller, there should be a GET endpoint that allows a logged-in user to view the list of attendees.
- The controller can delete events that the user attended, in case the user cannot attend the event anymore.

2.5. Make a new controller that allows a logged-in user to modify its attendance in the office.

- Endpoints are protected by a login.
- Endpoints are accessible by the user that is logged-in, the user may only edit its own attendance.
- You can re-use the code from the login service.
- API endpoint returns a reasonable error message If the user tries to book a date that is already occupied

2.6. Make a new controller that allows a logged-in user to book a room. (Younes)

- Endpoints are protected by a login.
- Endpoints are accessible by the user that is logged-in, the user may only book an available room.
- You can re-use the code from the login service.
- API endpoint returns a reasonable error message If the user tries to book a date that is already occupied

3.1. There must also be at least one custom feature implemented in the project to distinguish the application from others (Gavin)

- Decide with your team what you would like to do (if you'd like to learn more about front-end animation, think of a front-end heavy feature, for example), and create a clear description and requirements for this.
- The feature must first be proposed to your teacher and approved. Make sure you do this before you start implementing it.